JAVA PLACEMENT MASTER SERIES · HANDBOOK 4

Multithreading, JVM & Performance

Concurrency, Virtual Threads, JVM internals & Garbage Collection — the depth that separates senior candidates

★ synchronized, volatile & JMM

★ ExecutorService & CompletableFuture

★ Virtual Threads & G1 GC

★ Top 40 + Top 30 Interview Q&A

For MCA / B.Tech / B.E. Placement Preparation

Backend Roles · Performance & Concurrency Interviews

Table of Contents

Tip: If you're short on time, revise **Part 25 (Final Revision)** and **Part 13 (Virtual Threads)** first — concurrency + JVM fundamentals are what separate junior from senior-level answers in interviews.

How to use this handbook

This is **Handbook 4** of the Java Placement Master Series, covering Multithreading, JVM internals, Garbage Collection, and Performance — the topics that distinguish strong backend candidates in technical interviews. It covers concurrency primitives, the Executor Framework, CompletableFuture, Virtual Threads (Java 21), JVM architecture, GC internals, and the Java Memory Model. Read it top to bottom once, then use **Part 25 (Final Revision)** as your fast recap before interviews.

Importance tags used throughout:

- ⭐⭐⭐⭐⭐ Must Know for Placements
- ⭐⭐⭐⭐ Frequently Asked
- ⭐⭐ Medium Importance

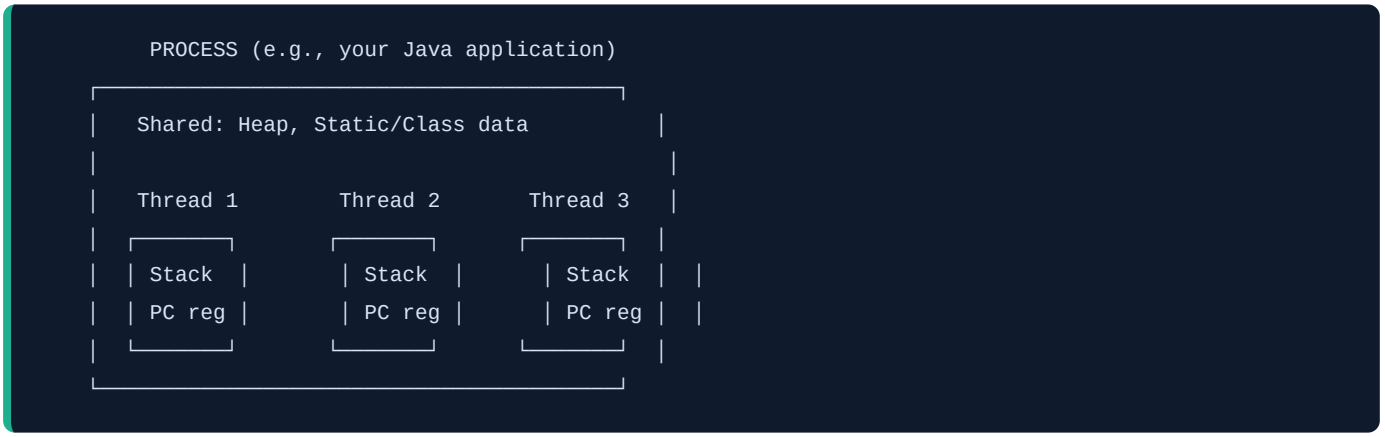
Part 1 — Process vs Thread ⭐⭐⭐⭐

What is a Process?

A **process** is an independent running program with its **own memory space** (code, data, heap, stack) — isolated from other processes by the OS. Starting a new process is expensive (OS must allocate a whole new memory space).

What is a Thread?

A **thread** is a lightweight unit of execution **within** a process. Multiple threads in the same process **share** the process's memory (heap, static data) but each has its **own stack** and program counter.



Real-world analogy: A process is like an entire **office building** (isolated, self-contained). Threads are like **employees working inside** the same building — they share the building's resources (meeting rooms = heap) but each has their own desk (stack) to work at independently.

Why Threads Are Lighter

Creating a thread only requires allocating a new stack + registers — much cheaper than a full process (new memory space, file handles, OS-level isolation). Thread creation and **context switching** (the CPU saving one thread's state and loading another's) are both significantly faster than the process equivalents.

Shared Memory & The Thread-Safety Problem ⭐⭐⭐⭐⭐

Because threads **share heap memory**, two threads can read/modify the **same object** at the same time — leading to unpredictable results if not properly coordinated. This is the root cause of nearly every concurrency bug, and the reason synchronization mechanisms (Part 3 onward) exist at all.

```
class Counter { int count = 0; }
// Thread A and Thread B both do: counter.count++;
// Both might read count=5 at the same time, both write count=6 -> one increment LOST!
```

Part 2 — Java Thread Basics ⭐⭐⭐⭐⭐

Creating Threads

```
// Option 1: extend Thread
class MyThread extends Thread {
    public void run() { System.out.println("Running"); }
}
new MyThread().start();      // start() creates a new OS thread and calls run() on it

// Option 2: implement Runnable (PREFERRED - decouples the task from thread mechanics)
Runnable task = () -> System.out.println("Running");
new Thread(task).start();
```

Why Runnable is preferred ⭐⭐⭐⭐⭐: Java has single inheritance — extending `Thread` uses up your only superclass slot. `Runnable` is just a task description; it can be handed to a `Thread`, an `ExecutorService`, or anything else that runs tasks, keeping the "what to do" separate from "how it's run."

Callable and Future ⭐⭐⭐⭐⭐

`Runnable.run()` returns nothing and can't throw checked exceptions. `Callable<V>` fixes both — it returns a value and can throw exceptions — but it can only be run via an `ExecutorService` (not a plain `Thread`), returning a `Future<V>` handle to retrieve the result later.

```
Callable<Integer> task = () -> { Thread.sleep(1000); return 42; };
ExecutorService executor = Executors.newSingleThreadExecutor();
Future<Integer> future = executor.submit(task);
Integer result = future.get();     // blocks until the result is ready
```

	`Runnable`	`Callable<V>`
Method	`void run()`	`V call()`
Returns a value	No	Yes
Can throw checked exceptions	No	Yes
Can run on plain `Thread`	Yes	No — needs `ExecutorService`

Naming Threads & Priorities

```
Thread t = new Thread(task, "WorkerThread-1");     // custom name, helps in debugging/logs
t.setPriority(Thread.MAX_PRIORITY);                   // hint only - scheduler isn't obligated to obey
it
```

Daemon Threads ⭐⭐⭐⭐

A **daemon thread** runs in the background and does **not** prevent the JVM from exiting — when all non-daemon (user) threads finish, the JVM shuts down even if daemon threads are still running (e.g., garbage collector threads).

```
Thread daemon = new Thread(task);
daemon.setDaemon(true);    // must be called BEFORE start()
daemon.start();
```

Thread Lifecycle ⭐⭐⭐⭐⭐

State	Meaning
`NEW`	Thread object created, `start()` not yet called
`RUNNABLE`	Eligible to run — either actually running or waiting for CPU time from the scheduler
`BLOCKED`	Waiting to acquire a lock held by another thread
`WAITING`	Waiting indefinitely for another thread's action (`wait()` , `join()` with no timeout)
`TIMED_WAITING`	Waiting for a bounded time (`sleep()` , `wait(timeout)`)
`TERMINATED`	`run()` has completed (normally or via exception)

Interview Questions — Part 1–2

Q1. Why is implementing Runnable preferred over extending Thread? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** Java doesn't support multiple inheritance, so extending `Thread` consumes your one superclass slot. Implementing `Runnable` keeps the task decoupled from the threading mechanism, letting the same task run on a plain `Thread` , an `ExecutorService` , or anywhere else — better design and reusability.
- **Difficulty:** Basic

Q2. Runnable vs Callable? ⭐⭐⭐⭐⭐

- **Ideal Answer:** `Runnable.run()` returns nothing and can't throw checked exceptions; it can run directly on a `Thread` . `Callable.call()` returns a value and can throw checked exceptions, but requires an `ExecutorService` and returns a `Future` to retrieve the result.
- **Difficulty:** Basic

Q3. What happens if you call run() directly instead of start()? ⭐⭐⭐⭐⭐

- **Ideal Answer:** Calling `run()` directly just executes it as a normal method call on the **current** thread — no new thread is created. Only `start()` actually creates a new OS-level thread and schedules `run()` to execute on it.
- **Why asked:** A classic trap question that catches candidates who don't understand what `start()` actually does.
- **Difficulty:** Intermediate

Q4. What is a daemon thread and when would you use one? ⭐⭐⭐⭐

- **Ideal Answer:** A background thread that doesn't block JVM shutdown — used for tasks like background monitoring, cleanup, or the garbage collector itself, where it's fine for the task to be abruptly terminated when the main application exits.
- **Difficulty:** Intermediate

Part 3 — Synchronization ⭐⭐⭐⭐⭐

Race Condition & Critical Section

A **race condition** occurs when multiple threads access shared data concurrently and the final result depends on unpredictable timing. The **critical section** is the part of code that touches shared data and must be protected so only one thread executes it at a time.

```
class Counter {
    int count = 0;
    void increment() { count++; }   // NOT atomic! read -> modify -> write = 3 separate steps
}
```

`count++` is actually **three steps**: read `count`, add 1, write back. If two threads interleave these steps, an increment can be lost.

`synchronized` Method and Block ⭐⭐⭐⭐⭐

```
class Counter {
    int count = 0;
    synchronized void increment() { count++; }        // synchronized METHOD - locks 'this'

    void incrementBlock() {
        synchronized (this) {                           // synchronized BLOCK - more granular control
            count++;
        }
    }
}
```

`synchronized` ensures only **one thread at a time** can execute the protected code for a given lock — other threads attempting to enter **block** (wait) until the lock is released.

Object Lock vs Class Lock ⭐⭐⭐⭐

- **Object lock (instance lock)** — `synchronized` instance methods/blocks lock on `this` (the specific object). Two threads calling a synchronized method on **two different objects** don't block each other.
- **Class lock** — `synchronized static` methods lock on the `Class` object itself (`ClassName.class`) — shared across **all** instances.

```
class Demo {
    synchronized void instanceMethod() { }        // locks on 'this' (per-object)
    static synchronized void staticMethod() { }     // locks on Demo.class (shared globally)
}
```

Reentrant Behavior ⭐⭐⭐⭐

`synchronized` locks in Java are **reentrant** — a thread already holding a lock can re-acquire the **same** lock again (e.g., calling another synchronized method on the same object from within a synchronized method) without deadlocking itself.

```
synchronized void outer() {
    inner();   // same thread re-entering a lock it already holds - OK, no deadlock
}
synchronized void inner() { }
```

Interview Questions — Part 3

Q1. What is a race condition, and how does synchronized fix it? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** A race condition happens when multiple threads access/modify shared data concurrently without coordination, producing unpredictable results. `synchronized` ensures mutual exclusion — only one thread can execute the protected critical section at a time, holding a lock that others must wait for.
- **Difficulty:** Basic

Q2. Object lock vs class lock? ⭐⭐⭐⭐

- **Ideal Answer:** An object lock (from instance-level `synchronized`) is per-object — different instances don't block each other. A class lock (from `static synchronized`) is shared across all instances since it locks on the `Class` object itself.
- **Difficulty:** Intermediate

Q3. Why are synchronized locks reentrant, and why does that matter? ⭐⭐⭐⭐

- **Ideal Answer:** Reentrancy lets a thread that already holds a lock re-acquire it without blocking itself — essential because a synchronized method commonly calls another synchronized method on the same object; without reentrancy, this would self-deadlock.
- **Difficulty:** Advanced

Part 4 — Communication Between Threads ⭐⭐⭐⭐

wait() / notify() / notifyAll()

These are `Object` class methods (not `Thread` methods!) used for threads to **coordinate** via a shared object's monitor. Must be called from within a `synchronized` block on that object.

Method	Effect
`wait()`	Releases the lock and pauses the thread until notified
`notify()`	Wakes up **one** waiting thread (unspecified which)
`notifyAll()`	Wakes up **all** waiting threads

wait() vs sleep() ⭐⭐⭐⭐⭐

	`wait()`	`sleep()`
Belongs to	`Object`	`Thread` (static method)
Releases the lock?	**Yes** — releases the monitor lock while waiting	**No** — keeps holding any locks it has
Wakes up via	`notify()` / `notifyAll()` (or timeout)	Timeout elapses
Must be in synchronized block	Yes	No

Producer-Consumer Example

```
class SharedBuffer {
    private final Queue<Integer> queue = new LinkedList<>();
    private final int capacity = 5;

    synchronized void produce(int value) throws InterruptedException {
        while (queue.size() == capacity) wait();     // full - wait for consumer to make room
        queue.add(value);
        notifyAll();                                  // wake up any waiting consumer
    }

    synchronized int consume() throws InterruptedException {
        while (queue.isEmpty()) wait();               // empty - wait for producer
        int value = queue.poll();
        notifyAll();                                  // wake up any waiting producer
        return value;
    }
}
```

Why `while`, not `if`, for the wait condition ⭐⭐⭐⭐⭐: Spurious wakeups — a thread can wake from `wait()` without actually being notified (a rare JVM-level possibility). Re-checking the condition in a `while` loop after waking ensures correctness even if the wakeup wasn't legitimate, or if another thread grabbed the resource first.

Interview Questions — Part 4

Q1. Why does wait() release the lock but sleep() doesn't? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** `wait()` is designed for cooperative coordination — a thread waits for some condition another thread will change, so it must release the lock, or no other thread could ever acquire it to make that change and call `notify()`. `sleep()` is just a timed pause unrelated to any lock/condition, so it has no reason to release anything.
- **Difficulty:** Intermediate

Q2. Why should the wait condition always be checked in a while loop, not an if? ⭐⭐⭐⭐⭐

- **Ideal Answer:** Spurious wakeups can occur (a thread resumes without an actual `notify()`), and even with a real notify, another thread might have already changed the condition before this thread gets scheduled. Re-checking in a `while` loop after waking guards against both cases.
- **Difficulty:** Advanced

Part 5 — Volatile Keyword ⭐⭐⭐⭐⭐

The Visibility Problem

Each CPU core may cache variables locally for speed. Without coordination, **Thread A's write to a shared variable might not be visible to Thread B** — B could keep reading a stale, cached value indefinitely.

```
Thread A (Core 1)              Thread B (Core 2)
  writes flag=true               reads flag from ITS OWN CPU cache
  (in CPU 1's cache,             (might still see the OLD value,
   not yet flushed to            even though A already updated it!)
   main memory)
```

What `volatile` Guarantees ⭐⭐⭐⭐⭐

```
private volatile boolean flag = false;
```

1. **Visibility** — every write to a `volatile` variable is immediately flushed to main memory, and every read fetches the latest value from main memory — no stale CPU-cache reads.
2. **Ordering (happens-before)** — the JVM won't reorder instructions around a volatile read/write in ways that would break this visibility guarantee.

What `volatile` Does NOT Guarantee ⭐⭐⭐⭐⭐

Atomicity. `volatile` does **not** make compound operations (like `count++`) atomic — that's still read-modify-write in 3 separate steps, and two threads can still race on it.

```
private volatile int count = 0;
void increment() { count++; }   // STILL NOT thread-safe! volatile ≠ atomic
```

Interview trap ⭐⭐⭐⭐⭐: `volatile` is perfect for a simple flag (`running`, `shutdownRequested`) that one thread sets and others read — but **wrong** for counters or anything requiring read-modify-write consistency. Use `AtomicInteger` / `synchronized` for those instead.

Use Case	Correct Tool
Simple flag, one writer, multiple readers	`volatile`
Counter, compound read-modify-write	`AtomicInteger` or `synchronized`
Complex multi-step invariant	`synchronized` or a `Lock`

Interview Questions — Part 5

Q1. What does volatile guarantee, and what doesn't it guarantee? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** It guarantees visibility (reads always see the latest write from any thread) and ordering (prevents certain instruction reordering). It does NOT guarantee atomicity — compound operations like `count++` are still multiple steps and can still race.
- **Why asked:** THE classic volatile trap question — separates real understanding from memorized keywords.
- **Difficulty:** Advanced

Q2. When would you use volatile instead of synchronized? ⭐⭐⭐⭐⭐

- **Ideal Answer:** When you have a simple flag or single variable with one writer and multiple readers, and no compound operations — `volatile` is cheaper (no locking overhead) and sufficient. For anything requiring atomic read-modify-write or multi-variable consistency, use `synchronized` or atomic classes instead.
- **Difficulty:** Advanced

Part 6 — Atomic Classes ⭐⭐⭐⭐

Why They Exist

`AtomicInteger`, `AtomicLong`, `AtomicReference`, etc. (from `java.util.concurrent.atomic`) provide **lock-free**, thread-safe operations on single variables — faster than `synchronized` for simple counters because they avoid the overhead of acquiring/releasing a lock.

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet();       // atomic - no race condition possible
counter.compareAndSet(5, 10);      // if current value is 5, set it to 10
```

Compare-And-Set (CAS) ⭐⭐⭐⭐⭐

The underlying mechanism: CAS is a single **hardware-level atomic instruction** that says *"update this value to X, but only if it currently equals Y — otherwise, tell me it failed."*

```
1. Read current value (say, 5)
2. Compute new value (say, 6)
3. CAS(expected=5, new=6):
     if memory still holds 5 → atomically write 6, succeed
     if memory now holds something else (another thread changed it) → FAIL, retry from step 1
```

This is **optimistic** concurrency — instead of locking upfront (pessimistic), it assumes no conflict, attempts the update, and retries only if another thread interfered. Under low contention, this is much faster than locking.

Interview Questions — Part 6

Q1. How does CAS work, and why is it faster than synchronized for simple counters? ⭐⭐⭐⭐⭐

- **Ideal Answer:** CAS is a hardware-supported atomic instruction that conditionally updates a value only if it still matches an expected value, retrying if not. It avoids the overhead of acquiring/releasing an OS-level lock — under typical low-to-moderate contention, this optimistic approach is significantly faster than `synchronized`'s pessimistic locking.
- **Difficulty:** Advanced

Part 7 — Executor Framework ⭐⭐⭐⭐⭐

Why Thread Pools Exist

Creating a new `Thread` for every task is expensive (OS-level allocation) and uncontrolled (nothing stops you from spawning thousands of threads and crashing the app). A **thread pool** maintains a fixed set of **reusable** worker threads that pick up tasks from a queue — controlled resource usage, no per-task thread creation cost.

```
Task Queue: [T1] [T2] [T3] [T4] [T5] ...
              |
              ▼
      ┌──────────────────────┐
      |   Thread Pool (size 3) |
      |  Worker1  Worker2  Worker3 |
      └──────────────────────┘
     Each worker picks the next task once free - threads are REUSED, not recreated
```

Executor / ExecutorService / submit vs execute

```
ExecutorService executor = Executors.newFixedThreadPool(4);

executor.execute(() -> System.out.println("Runnable, no result"));   // Runnable, returns void

Future<Integer> future = executor.submit(() -> 42);                    // Runnable/Callable, returns a Future

executor.shutdown();       // graceful: finishes queued/running tasks, rejects new ones
executor.shutdownNow();     // aggressive: attempts to interrupt running tasks, drains the queue
```

	`execute()`	`submit()`
Accepts	`Runnable` only	`Runnable` or `Callable`
Returns	Nothing	`Future<T>`
Exception handling	Uncaught exceptions go to the thread's uncaught exception handler	Exceptions are captured in the `Future`, surfaced when `.get()` is called

Thread Pool Types ⭐⭐⭐⭐⭐

Factory Method	Behavior	Best For
`newFixedThreadPool(n)`	Fixed number of threads, unbounded queue	Predictable, steady workloads
`newCachedThreadPool()`	Creates threads as needed, reuses idle ones, no upper bound	Many short-lived, bursty tasks
`newSingleThreadExecutor()`	Exactly one worker thread, tasks run sequentially	Tasks that must execute strictly in order
`newScheduledThreadPool(n)`	Supports delayed/periodic task execution	Cron-like recurring jobs

Interview trap ⭐⭐⭐⭐: `newCachedThreadPool()` has **no upper bound** on thread count — under heavy sustained load, it can create an unbounded number of threads and exhaust system resources. In production, a custom `ThreadPoolExecutor` with an explicit bounded queue and max pool size is usually safer.

Interview Questions — Part 7

Q1. submit() vs execute()? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** `execute()` takes only a `Runnable` and returns nothing — exceptions propagate to the thread's uncaught exception handler. `submit()` accepts `Runnable` or `Callable` and returns a `Future`, capturing exceptions so they surface when `.get()` is called instead of being silently lost.
- **Difficulty:** Intermediate

Q2. Why is newCachedThreadPool dangerous under heavy load? ⭐⭐⭐⭐⭐

- **Ideal Answer:** It has no maximum thread count — if tasks arrive faster than they complete, it keeps spawning new threads indefinitely, which can exhaust memory/OS thread limits and crash the application. A bounded custom `ThreadPoolExecutor` is safer for production use.
- **Difficulty:** Advanced

Q3. shutdown() vs shutdownNow()? ⭐⭐⭐⭐

- **Ideal Answer:** `shutdown()` stops accepting new tasks but lets already-submitted tasks (including queued ones) finish. `shutdownNow()` attempts to stop all actively executing tasks immediately (via interruption) and returns the list of tasks that were still waiting in the queue.
- **Difficulty:** Intermediate

Part 8 — Locks API ⭐⭐⭐⭐⭐

ReentrantLock vs synchronized

```
private final ReentrantLock lock = new ReentrantLock();

void increment() {
    lock.lock();
    try {
        count++;
    } finally {
        lock.unlock();     // MUST be in finally - or a lock leak occurs on exception!
    }
}
```

	`synchronized`	`ReentrantLock`
Lock acquisition	Implicit (block/method entry)	Explicit `lock()` / `unlock()` calls
Try without blocking	Not possible	`tryLock()` — attempt, don't block if unavailable
Timed attempt	Not possible	`tryLock(timeout, unit)`
Fairness (FIFO ordering)	Not supported	`new ReentrantLock(true)` for a fair lock
Interruptible waiting	Not possible	`lockInterruptibly()`
Auto-release on exception	Yes, automatic	**No** — must manually `unlock()` in `finally`

When to use `ReentrantLock` over `synchronized` ⭐⭐⭐⭐: when you need `tryLock()` (avoid blocking indefinitely), fairness guarantees, or interruptible lock acquisition — otherwise, `synchronized` is simpler and less error-prone (no risk of forgetting to unlock).

ReadWriteLock ⭐⭐⭐⭐

Allows **multiple concurrent readers**, but only **one exclusive writer** (and no readers while writing) — much better throughput than a single exclusive lock for read-heavy workloads.

```
ReadWriteLock rwLock = new ReentrantReadWriteLock();
rwLock.readLock().lock();    // many threads can hold the read lock simultaneously
rwLock.writeLock().lock();    // only one thread, and no concurrent readers, during a write
```

StampedLock (overview) ⭐⭐⭐

An even more advanced lock offering an **optimistic read** mode — readers don't block writers at all; they read speculatively and then validate whether a write happened concurrently, retrying with a full read lock if so. Higher throughput than `ReadWriteLock` in read-heavy scenarios, but more complex and not reentrant — used mainly in performance-critical libraries.

Interview Questions — Part 8

Q1. Why would you choose ReentrantLock over synchronized? ⭐⭐⭐⭐⭐

- **Ideal Answer:** When you need capabilities `synchronized` doesn't offer: `tryLock()` for non-blocking attempts, timed lock attempts, fairness ordering, or interruptible lock waits. Otherwise `synchronized` is preferred for its simplicity and automatic unlock safety.
- **Difficulty:** Intermediate

Q2. What is the biggest risk when using ReentrantLock? ⭐⭐⭐⭐⭐

- **Ideal Answer:** Forgetting to call `unlock()` — unlike `synchronized`, which releases automatically even on exception, `ReentrantLock` requires explicit `unlock()` in a `finally` block, or the lock leaks and can deadlock other threads permanently.
- **Difficulty:** Intermediate

Part 9 — Concurrent Collections ⭐⭐⭐⭐⭐

ConcurrentHashMap ⭐⭐⭐⭐⭐

The go-to thread-safe map. Unlike `Hashtable` (locks the **entire** map per operation), `ConcurrentHashMap` uses **fine-grained locking** — in Java 8+, each bucket is independently synchronized (via `synchronized` blocks on the bucket's first node, plus CAS operations), so multiple threads can read/write **different** buckets concurrently with minimal contention.

```
ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();
map.put("A", 1);
map.computeIfAbsent("B", k -> 0);        // atomic - safe under concurrent access
map.merge("A", 1, Integer::sum);          // atomic increment pattern
```

Disallows `null` keys/values (ambiguity risk in concurrent reads — can't tell "absent" from "mapped to null").

CopyOnWriteArrayList ⭐⭐⭐⭐

Every **write** (add/remove/set) creates a **brand-new copy** of the entire underlying array. Reads never block and never see `ConcurrentModificationException`, because iterators work off a fixed snapshot.

```
write() → copy entire array → modify the copy → swap the reference atomically
```

Best for: read-heavy, write-rare scenarios (e.g., a list of event listeners). **Bad for:** write-heavy use — every write is O(n) due to the full copy.

BlockingQueue ⭐⭐⭐⭐⭐

A queue where `put()` **blocks** if the queue is full, and `take()` **blocks** if the queue is empty — the foundation of the classic **Producer-Consumer** pattern.

```
BlockingQueue<Task> queue = new LinkedBlockingQueue<>(100);
// Producer thread:
queue.put(new Task());     // blocks if queue is full
// Consumer thread:
Task t = queue.take();      // blocks if queue is empty
```

This is exactly how `ThreadPoolExecutor` internally queues tasks waiting for a free worker thread.

ConcurrentLinkedQueue ⭐⭐⭐

A **non-blocking**, lock-free queue based on CAS operations — high throughput for producer-consumer patterns that don't need blocking semantics (callers must poll/retry rather than block).

When to Use Which

Need	Use
Thread-safe general-purpose map	`ConcurrentHashMap`
Read-heavy list, rare writes	`CopyOnWriteArrayList`
Producer-consumer with blocking	`BlockingQueue` (e.g., `LinkedBlockingQueue`)
High-throughput non-blocking queue	`ConcurrentLinkedQueue`

Interview Questions — Part 9

Q1. How does ConcurrentHashMap achieve thread safety without locking the whole map? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** It uses fine-grained locking at the bucket level (synchronized blocks on individual bucket nodes) combined with CAS operations for many operations, allowing multiple threads to operate on different buckets simultaneously — far better concurrent throughput than `Hashtable`'s single full-map lock.
- **Difficulty:** Advanced

Q2. When would CopyOnWriteArrayList hurt performance? ⭐⭐⭐⭐

- **Ideal Answer:** In write-heavy scenarios — every single write copies the entire backing array (O(n)), so frequent modifications on a large list become very expensive. It's only suitable for read-heavy, write-rare use cases.

- **Difficulty:** Intermediate

Part 10 — Deadlock, Livelock, Starvation ⭐⭐⭐⭐⭐

How Deadlock Occurs

Two (or more) threads each hold a lock the other needs, and neither will release theirs — both wait **forever**.

```
Thread A: locks Resource 1 ──► wants Resource 2 (held by B) ──► WAITS
Thread B: locks Resource 2 ──► wants Resource 1 (held by A) ──► WAITS
                  │                                    │
                  └──────────── DEADLOCK ──────────────┘
```

```
// Classic deadlock
Object lock1 = new Object(), lock2 = new Object();

// Thread 1:
synchronized (lock1) {
    synchronized (lock2) { /* ... */ }
}
// Thread 2 (locks acquired in OPPOSITE order):
synchronized (lock2) {
    synchronized (lock1) { /* ... */ }
}
```

Four Necessary Conditions (Coffman Conditions) ⭐⭐⭐⭐⭐

Condition	Meaning
Mutual Exclusion	A resource can be held by only one thread at a time
Hold and Wait	A thread holds a resource while waiting for another
No Preemption	A resource can't be forcibly taken from a thread — only voluntarily released
Circular Wait	A cycle of threads, each waiting for a resource held by the next

Deadlock requires ALL four simultaneously — breaking any one prevents it.

Prevention Techniques

- **Lock ordering** ⭐⭐⭐⭐⭐ — always acquire locks in the same **global order** across all threads, eliminating circular wait.
- `tryLock()` **with timeout** — back off and retry instead of waiting forever.
- **Minimize lock scope** — hold locks for the shortest time possible, reducing overlap opportunities.
- **Avoid nested locks** where possible.

Livelock vs Starvation

Deadlock	Livelock	Starvation
Threads **stuck**, doing nothing	Threads **actively responding** to each other, but making no real progress	A thread **never** gets CPU time/lock access due to unfair scheduling
e.g., circular lock wait	e.g., two threads repeatedly "politely" backing off for each other	e.g., low-priority thread starved by constant high-priority threads

Interview Questions — Part 10

Q1. What are the four necessary conditions for deadlock? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** Mutual exclusion, hold-and-wait, no preemption, and circular wait — all four must hold simultaneously for deadlock to occur; breaking any single one prevents it.
- **Difficulty:** Intermediate

Q2. How would you prevent deadlock in practice? ⭐⭐⭐⭐⭐

- **Ideal Answer:** The most reliable technique is enforcing a consistent global lock acquisition order across all threads, eliminating circular wait. Alternatives include using `tryLock()` with timeouts to back off instead of blocking forever, and minimizing the scope/duration of held locks.
- **Difficulty:** Intermediate

Q3. Scenario: Thread A locks resource X then Y; Thread B locks Y then X. How do you fix this? ⭐⭐⭐⭐⭐

- **Ideal Answer:** Change one of the threads to acquire locks in the same order as the other (e.g., always X before Y) — this eliminates the circular wait condition and prevents the deadlock entirely.
- **Difficulty:** Intermediate

Part 11 — CompletableFuture ⭐⭐⭐⭐⭐

Why It Was Introduced

A plain `Future` can only **block** on `.get()` — no way to chain follow-up actions, combine multiple futures, or react to completion asynchronously. `CompletableFuture` (Java 8+) supports composable, non-blocking **asynchronous pipelines**.

```
CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() -> {
    return 10 + 20;                    // runs on the common ForkJoinPool by default
});

CompletableFuture<Void> task = CompletableFuture.runAsync(() -> System.out.println("Done"));
```

Chaining — thenApply, thenCompose, thenCombine ⭐⭐⭐⭐⭐

```
// thenApply - transform the result (like Stream.map)
CompletableFuture<Integer> doubled = CompletableFuture.supplyAsync(() -> 5)
    .thenApply(n -> n * 2);       // 10

// thenCompose - chain another ASYNC operation (like Stream.flatMap) - avoids nested futures
CompletableFuture<String> chained = CompletableFuture.supplyAsync(() -> 5)
    .thenCompose(n -> CompletableFuture.supplyAsync(() -> "Result: " + n));

// thenCombine - combine TWO independent futures once both complete
CompletableFuture<Integer> a = CompletableFuture.supplyAsync(() -> 10);
CompletableFuture<Integer> b = CompletableFuture.supplyAsync(() -> 20);
CompletableFuture<Integer> sum = a.thenCombine(b, Integer::sum);   // 30
```

```
supplyAsync() ──► thenApply() ──► thenCompose() ──► thenCombine() ──► result
   (start)          (transform)      (chain async)     (merge two)
```

Error Handling

```
CompletableFuture<Integer> safe = CompletableFuture.supplyAsync(() -> 10 / 0)
    .exceptionally(ex -> {
        System.out.println("Error: " + ex.getMessage());
        return -1;                  // fallback value
    });
```

allOf and anyOf

```
CompletableFuture<Void> all = CompletableFuture.allOf(a, b);     // completes when BOTH finish
CompletableFuture<Object> any = CompletableFuture.anyOf(a, b);    // completes when EITHER finishes
first
```

Interview Questions — Part 11

Q1. Why is CompletableFuture better than a plain Future? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** A plain `Future` only supports blocking `.get()` — no chaining, no composition, no reaction to completion. `CompletableFuture` supports non-blocking callbacks (`thenApply`, `thenCompose`), combining multiple futures (`thenCombine`, `allOf`), and explicit exception handling (`exceptionally`), enabling full asynchronous pipelines.
- **Difficulty:** Intermediate

Q2. thenApply vs thenCompose? ⭐⭐⭐⭐⭐

- **Ideal Answer:** `thenApply` transforms a result synchronously within the pipeline (like `Stream.map`). `thenCompose` is used when the transformation itself returns another `CompletableFuture` — it "flattens" the result instead of producing a nested `CompletableFuture<CompletableFuture<T>>` (like `Stream.flatMap`).
- **Difficulty:** Advanced

Part 12 — Parallel Streams ⭐⭐⭐⭐

How They Work

`.parallelStream()` splits the source data into chunks and processes them concurrently using the shared **common** `ForkJoinPool` (default size = number of CPU cores - 1), then merges the results.

```
int sum = List.of(1,2,3,4,5).parallelStream()
    .mapToInt(Integer::intValue)
    .sum();
```

When to Use / When NOT To ⭐⭐⭐⭐⭐

Use parallel streams when...	Avoid parallel streams when...
Large datasets (thousands+ elements)	Small collections — overhead exceeds benefit
CPU-heavy, independent operations	I/O-bound operations (blocking wastes pooled threads)
No shared mutable state between elements	Operations have side effects on shared state (race conditions!)
Order doesn't matter (or `forEachOrdered` is acceptable)	Strict ordering is required (adds overhead)

Danger ⭐⭐⭐⭐⭐: All parallel streams in an application share the **same** common `ForkJoinPool`. A single slow/blocking parallel stream operation can starve unrelated parallel work elsewhere in the app.

Interview Questions — Part 12

Q1. What thread pool do parallel streams use, and why does that matter? ⭐⭐⭐⭐⭐

- **Ideal Answer:** The shared common `ForkJoinPool`, sized to the number of CPU cores by default. Because it's shared across the whole application, a blocking or long-running task in one parallel stream can starve other unrelated parallel streams competing for the same pool.
- **Difficulty:** Advanced

Part 13 — Virtual Threads (Java 21) ⭐⭐⭐⭐⭐

Platform Threads vs Virtual Threads

Platform threads map 1:1 to **OS threads** — expensive to create (each reserves significant stack memory, ~1MB+), and the OS can only handle a limited number (thousands) before performance collapses.

Virtual threads are lightweight threads **managed by the JVM**, not the OS — millions can exist simultaneously. Many virtual threads are multiplexed onto a small number of platform "carrier" threads.

```
Platform Threads:                    Virtual Threads:
Thread ──► OS Thread (1:1)           Thread ──► JVM Scheduler ──► Carrier Thread (M:N)
                                                (many virtual threads share few OS threads)

  OS Thread 1 ── Platform Thread A       Carrier Thread 1 ── VT1, VT2, VT3...
  OS Thread 2 ── Platform Thread B       Carrier Thread 2 ── VT4, VT5, VT6...
  (limited to ~thousands)                (millions possible)
```

The Key Trick: Blocking Doesn't Waste an OS Thread ⭐⭐⭐⭐⭐

When a virtual thread performs a **blocking** operation (like I/O), the JVM automatically **unmounts** it from its carrier thread, freeing that carrier to run other virtual threads. When the blocking operation completes, the virtual thread is **remounted** onto some available carrier thread to continue.

```
Thread.startVirtualThread(() -> {
    System.out.println("Running on: " + Thread.currentThread());
});

// Or via ExecutorService (recommended way to manage many virtual threads)
try (ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 100_000; i++) {
        executor.submit(() -> {
            Thread.sleep(Duration.ofSeconds(1));   // doesn't block the underlying OS thread!
            return null;
        });
    }
}
```

Comparison Table

	Platform Thread	Virtual Thread
Managed by	OS	JVM
Memory per thread	~1MB+ stack	A few KB, grows as needed
Max practical count	Thousands	Millions
Blocking I/O cost	Wastes an OS thread while blocked	Carrier thread freed for other work
Creation cost	Expensive	Very cheap
Best for	CPU-bound work	High-concurrency I/O-bound work (e.g., many simultaneous DB/HTTP calls)

Limitations ⭐⭐⭐⭐

- **`synchronized` blocks pin the carrier thread** — a virtual thread inside a `synchronized` block that then blocks does NOT unmount, defeating the benefit (use `ReentrantLock` instead in virtual-thread-heavy code, when possible).
- Not faster for **CPU-bound** work — virtual threads help massively with I/O-bound concurrency, not raw computation speed.
- Thread-local heavy code can behave unexpectedly at extreme scale (millions of threads means millions of thread-locals).

Interview Questions — Part 13

Q1. What problem do Virtual Threads solve? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** Traditional platform threads map 1:1 to OS threads, which are expensive and limited to roughly thousands per application. Virtual threads are JVM-managed and lightweight, allowing millions of concurrent tasks — ideal for high-concurrency I/O-bound workloads like handling many simultaneous HTTP requests, without the memory/scheduling cost of that many OS threads.
- **Difficulty:** Intermediate

Q2. Why doesn't blocking I/O waste resources with virtual threads? ⭐⭐⭐⭐⭐

- **Ideal Answer:** When a virtual thread blocks on I/O, the JVM automatically unmounts it from its carrier (platform) thread, freeing that carrier to run other virtual threads. When the I/O completes, the virtual thread remounts onto an available carrier to resume — so blocking a virtual thread doesn't block an underlying OS thread.
- **Difficulty:** Advanced

Q3. What is the "pinning" problem with virtual threads? ⭐⭐⭐⭐

- **Ideal Answer:** If a virtual thread blocks while inside a `synchronized` block/method, it cannot unmount from its carrier thread (this is a JVM implementation constraint) — it "pins" the carrier, blocking it from running other virtual threads for that duration. Using `ReentrantLock` instead of `synchronized` avoids this in virtual-thread-heavy code.
- **Why asked:** A very current, high-signal question showing awareness of real Java 21 gotchas.
- **Difficulty:** Advanced

Q4. Are virtual threads good for CPU-intensive work? ⭐⭐⭐⭐

- **Ideal Answer:** No — virtual threads solve the cost of *blocking/waiting* (I/O-bound concurrency), not computation speed. For CPU-bound work, the number of platform/carrier threads (tied to CPU core count) is still the real bottleneck.
- **Difficulty:** Intermediate

Part 14 — JVM Architecture

Runtime Data Areas

Area	Shared/Per-thread	Purpose
Heap	Shared across all threads	All objects and arrays live here; managed by the Garbage Collector
Stack	Per-thread	Stores stack frames — local variables, method call/return info, partial results
Metaspace	Shared	Class metadata (method bytecode, field info, runtime constant pool) — replaced PermGen in Java 8+, grows into **native** memory (not the heap)
PC (Program Counter) Register	Per-thread	Tracks the address of the JVM instruction currently executing for that thread
Native Method Stack	Per-thread	Supports native (non-Java, e.g., C/C++ via JNI) method calls

Execution Engine

- **Interpreter** — executes bytecode line by line; starts immediately but is slower for repeated execution.
- **JIT Compiler** — detects "hot" (frequently executed) methods and compiles them to native machine code at runtime for near-native speed on subsequent calls.
- **Garbage Collector** — automatically reclaims heap memory no longer reachable (Part 17).

Why Metaspace instead of PermGen? ⭐⭐⭐⭐: PermGen (pre-Java 8) had a **fixed** maximum size, causing `OutOfMemoryError: PermGen space` in apps that loaded many classes (common with app servers/frameworks doing lots of dynamic class generation). Metaspace uses native memory and **grows automatically** (up to available system memory, or a configured limit), largely eliminating that class of error.

Part 15 — Class Loading ⭐⭐⭐⭐

The Full Sequence

```
Loading ⟶ Linking (Verification → Preparation → Resolution) ⟶ Initialization
```

Phase	What Happens
Loading	Reads the `.class` file's bytecode and creates a `Class` object in memory
Verification	Checks the bytecode is structurally valid and doesn't violate JVM safety rules (prevents malicious/corrupted bytecode from running)
Preparation	Allocates memory for **static fields**, sets them to their **default** values (0, null, false — NOT their initializer values yet)
Resolution	Resolves symbolic references (class/method/field names) into direct references
Initialization	Runs static initializers and static variable assignments, **in the order they appear in the source code**

Static Initialization Order Example ⭐⭐⭐⭐

```
class Demo {
    static int a = 10;                          // 1st: assigned 10
    static { System.out.println("Block: " + a); }  // 2nd: static block, runs in source order → prints
10
    static int b = a + 5;                          // 3rd: b = 15
}
```

Static fields/blocks execute **top-to-bottom, in the exact order they appear in the source file** — a common output-prediction interview trap.

Interview Questions — Part 14–15

Q1. What replaced PermGen, and why? ⭐⭐⭐⭐⭐

- **Ideal Answer:** Metaspace (Java 8+). PermGen had a fixed max size and commonly caused `OutOfMemoryError: PermGen space` in apps loading many classes. Metaspace uses native memory and grows automatically, largely eliminating that specific error.
- **Difficulty:** Intermediate

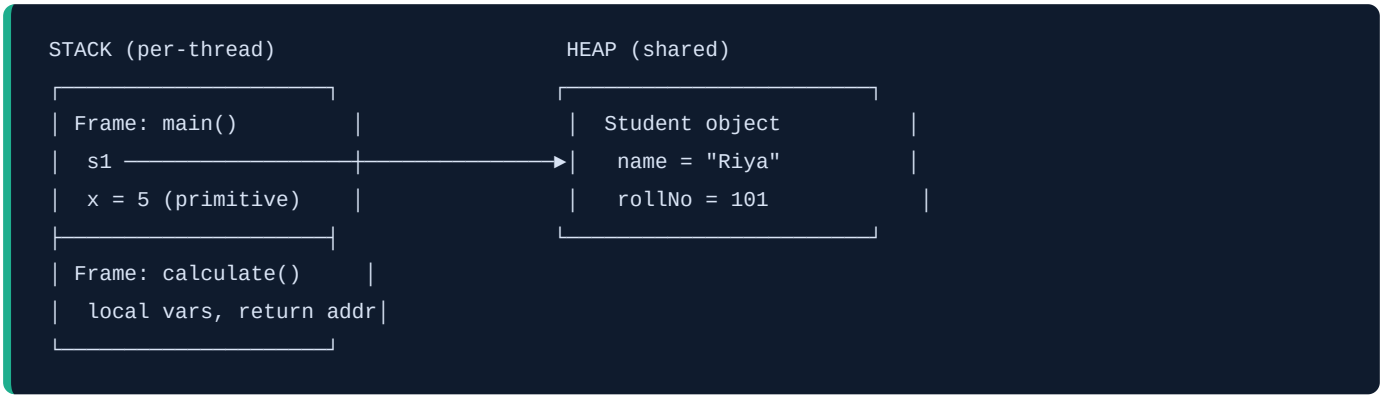
Q2. What happens during the Preparation phase of class loading? ⭐⭐⭐⭐

- **Ideal Answer:** Memory is allocated for static fields and they're set to their **default** values (0/null/false) — NOT their actual initializer expressions, which only run later during Initialization.
- **Difficulty:** Advanced

Q3. In what order do static blocks and static variable initializers run? ⭐⭐⭐⭐

- **Ideal Answer:** In the exact top-to-bottom order they appear in the source code, during the Initialization phase of class loading.
- **Difficulty:** Intermediate

Part 16 — Heap & Stack ⭐⭐⭐⭐⭐



- **Local variables and primitives** live directly on the **stack**, inside that method's **stack frame**.
- **Objects** always live on the **heap** — a stack variable only ever holds a **reference** (address) to a heap object, never the object itself.
- Every method call pushes a new **stack frame**; when the method returns, its frame is popped and all its local variables/references disappear immediately.
- **Object allocation:** `new` always allocates on the heap, no exceptions (ignoring rare JIT escape-analysis optimizations that can stack-allocate objects proven never to escape a method — an internal optimization, not something you control directly).

GC Roots ⭐⭐⭐⭐⭐

GC Roots are the starting points the Garbage Collector uses to determine what's "reachable" (alive). Anything **not reachable** from a GC root (directly or transitively) is garbage.

GC Root Examples
Local variables/parameters on any thread's active stack
Active threads themselves
Static fields of loaded classes
JNI references (native code holding a Java object)

```
GC Roots ──▶ reachable objects (ALIVE, kept)
   │
   └─▶ object with no path from any GC root = GARBAGE (eligible for collection)
```

Interview Questions — Part 16

Q1. What determines whether an object is garbage-collected? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** Reachability from a GC Root — if there's no chain of references from any GC Root (stack locals, static fields, active threads, JNI refs) to an object, it's unreachable and eligible for garbage collection, regardless of whether it was recently

used.

- **Difficulty:** Intermediate

Q2. Why does a StackOverflowError happen, and where — heap or stack? ⭐⭐⭐⭐⭐

- **Ideal Answer:** It happens on the **stack** — typically from unterminated/excessive recursion, where each call pushes a new stack frame until the fixed-size stack is exhausted. It's unrelated to heap memory.
- **Difficulty:** Basic

Q3. If a stack variable holds a reference to a heap object, and the method returns, what happens to the object? ⭐⭐⭐⭐

- **Ideal Answer:** The stack frame (and that local reference) is destroyed when the method returns. If no other reference to the object exists anywhere reachable from a GC root, the object becomes eligible for garbage collection; if another reference still exists (e.g., it was returned or stored elsewhere), the object remains alive.
- **Difficulty:** Intermediate

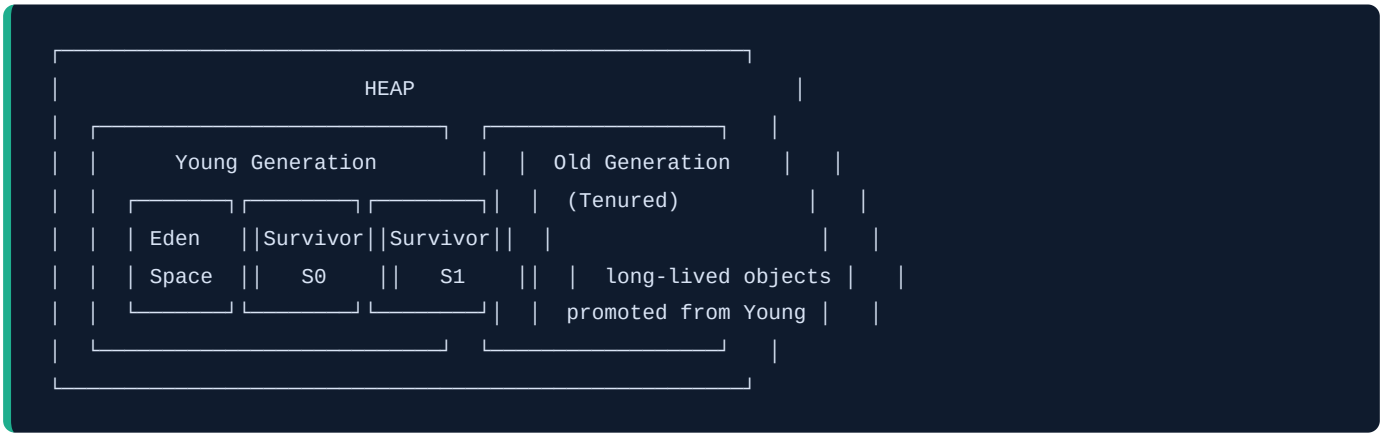
Part 17 — Garbage Collection ⭐⭐⭐⭐⭐

Why GC Exists

Manual memory management (like C/C++'s `malloc` / `free`) is a major source of bugs — memory leaks (forgetting to free) and dangling pointers (freeing too early). The JVM's **Garbage Collector** automatically reclaims memory for objects that are no longer **reachable**, eliminating both classes of bugs.

Generational Hypothesis & Heap Layout ⭐⭐⭐⭐⭐

Observation: **most objects die young** (e.g., temporary variables, request-scoped objects). The heap is split into generations to exploit this:



- New objects are allocated in **Eden**. When Eden fills up, a **Minor GC** runs.
- **Minor GC** — collects the Young Generation only. Surviving objects move to a Survivor space (S0/S1 alternate), and objects that survive several Minor GCs get **promoted** to the Old Generation. Minor GCs are frequent but fast.
- **Major GC** — collects the Old Generation.
- **Full GC** — collects the **entire** heap (Young + Old, sometimes Metaspace too) — much slower, causes longer application pauses; a common performance-tuning target.

Collectors — Interview-Level Overview

Collector	Strategy	Best For
Serial	Single-threaded, stop-the-world	Small applications, single-core environments
Parallel	Multi-threaded, stop-the-world	Throughput-focused batch jobs, willing to accept longer pauses for higher overall throughput
G1 (Garbage First) ⭐⭐⭐⭐⭐	Divides heap into many small regions; collects the regions with the most garbage first	**Default collector since Java 9** — balances throughput and low pause times for most general-purpose applications
ZGC ⭐⭐⭐⭐	Concurrent, extremely low pause times (sub-millisecond), scales to very large heaps (TBs)	Latency-critical applications with huge heaps
Shenandoah ⭐⭐⭐	Similar goal to ZGC — concurrent, low-pause collection	Alternative low-latency collector (Red Hat-developed)

G1 in a bit more depth ⭐⭐⭐⭐⭐: Instead of one contiguous Young/Old split, G1 divides the heap into many equal-sized **regions**, each independently tagged as Eden, Survivor, or Old. It prioritizes collecting regions with the most reclaimable garbage first ("Garbage First") — giving predictable, configurable pause-time targets instead of the older collectors' more unpredictable stop-the-world pauses.

Interview Questions — Part 17

Q1. What is the difference between Minor GC, Major GC, and Full GC? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** Minor GC collects only the Young Generation (frequent, fast). Major GC collects the Old Generation. Full GC collects the entire heap (and often Metaspace), causing the longest application pauses — a key performance tuning target.
- **Difficulty:** Intermediate

Q2. Why is the heap divided into generations? ⭐⭐⭐⭐⭐

- **Ideal Answer:** Based on the generational hypothesis — most objects die young. Frequently collecting just a small Young Generation is far cheaper than scanning the whole heap every time, while long-lived objects are promoted to the Old Generation, which is collected less often.
- **Difficulty:** Intermediate

Q3. Why is G1 the default collector since Java 9? ⭐⭐⭐⭐⭐

- **Ideal Answer:** G1 divides the heap into many small regions and prioritizes collecting the ones with the most garbage first, offering a good balance of throughput and predictable, configurable pause times — suitable as a general-purpose default for most applications, unlike Serial (single-threaded) or Parallel (longer pauses) which suit narrower use cases.
- **Difficulty:** Advanced

Q4. When would you choose ZGC over G1? ⭐⭐⭐⭐

- **Ideal Answer:** When ultra-low pause times (sub-millisecond) matter more than raw throughput, especially with very large heaps — e.g., latency-sensitive trading systems or large in-memory caches.
- **Difficulty:** Advanced

Part 18 — Memory Problems ⭐⭐⭐⭐

Problem	Cause	Prevention/Detection
Memory Leak	Objects stay unintentionally **reachable** (so GC can't collect them) even though the app no longer needs them — e.g., objects held in a growing static collection, unclosed resources, forgotten listeners	Use heap dump analysis (e.g., Eclipse MAT), review long-lived collections/caches for unbounded growth, prefer weak references for caches where appropriate
OutOfMemoryError: Java heap space	The heap is full and GC can't reclaim enough to satisfy an allocation	Increase `-Xmx`, fix leaks, review large object allocations/caches
StackOverflowError	Excessive/unterminated recursion exhausts a thread's stack	Add/fix recursion base cases; consider converting to an iterative approach for deep recursion
ClassLoader Leaks	A ClassLoader (and everything it loaded) can't be garbage collected because something still references one of its loaded classes/objects — classic issue in app servers doing hot redeployment	Ensure static references, thread-locals, and listeners registered by a webapp are cleaned up on undeploy
Collection Leaks	Continuously adding to a `static` or long-lived collection without ever removing old entries	Use bounded caches (e.g., `LinkedHashMap` LRU eviction, or a proper caching library), review collection lifecycle
Thread Leaks	Threads/thread pools created but never properly shut down, accumulating over time	Always call `executor.shutdown()`; use try-with-resources for `AutoCloseable` executors (Java 19+); monitor thread counts

Interview Questions — Part 18

Q1. How can a memory leak happen in Java despite automatic garbage collection? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** GC only reclaims **unreachable** objects. If an object is still referenced somewhere reachable (e.g., a growing static `List` that's never cleared, or a listener that's registered but never unregistered), it stays "alive" from the GC's perspective forever, even if the application logically no longer needs it — this is a Java-style memory leak.
- **Difficulty:** Intermediate

Q2. What's a common real-world cause of ClassLoader leaks? ⭐⭐⭐⭐

- **Ideal Answer:** Web application hot-redeployment — if a thread, static field, or ThreadLocal created by the old deployment's classes is still referenced anywhere (e.g., by a container-level object), the entire old ClassLoader and all its loaded classes can't be collected, leading to Metaspace/memory bloat over repeated redeploys.
- **Difficulty:** Advanced

Part 19 — Java Memory Model (JMM) ⭐⭐⭐⭐⭐

The Core Problem: Visibility, Ordering, Atomicity

In a multithreaded, multi-core system, each CPU core may cache variables **locally** — a write by Thread A isn't automatically, immediately visible to Thread B without explicit synchronization.

```
Thread A (Core 1)          Thread B (Core 2)
  writes x = 1                reads x
  (in CPU cache,               (might still see the OLD
   not yet flushed              cached value, or even 0!)
   to main memory)
```

The **Java Memory Model** defines the rules for when writes by one thread are **guaranteed** to be visible to another thread — this guarantee is called **happens-before**.

Happens-Before Relationship ⭐⭐⭐⭐⭐

If action A **happens-before** action B, then A's effects (including memory writes) are guaranteed visible to B. Key happens-before rules:

Rule	Guarantee
Program order	Within a single thread, each action happens-before the next one in code order
Monitor lock	An `unlock` happens-before every subsequent `lock` on the **same** monitor (by any thread)
Volatile	A write to a `volatile` field happens-before every subsequent read of that same field
Thread start	`Thread.start()` happens-before any action in the started thread
Thread join	Every action in a thread happens-before another thread successfully returns from `join()` on it

What Synchronization Provides ⭐⭐⭐⭐⭐

`synchronized` and `volatile` don't just prevent race conditions — they establish **happens-before** edges, guaranteeing both:

1. **Visibility** — writes before a lock release/volatile write are visible after the corresponding lock acquire/volatile read.
2. **Ordering** — the compiler/CPU cannot reorder instructions across these boundaries in ways that would violate the happens-before guarantee.

```
class SharedFlag {
    private volatile boolean ready = false;   // volatile ensures visibility across threads
    void writer() { ready = true; }             // this write is guaranteed visible...
    void reader() { while (!ready) {} }          // ...to this read, once it observes 'true'
}
```

Interview Questions — Part 19

Q1. What is "happens-before" and why does it matter? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** It's the JMM's formal guarantee that if action A happens-before action B, A's memory effects are visible to B. It matters because, without such a guarantee, one thread's writes might never become visible to another thread (due to CPU caching/instruction reordering), causing subtle, hard-to-reproduce concurrency bugs.
- **Difficulty:** Advanced

Q2. Does synchronized only provide mutual exclusion, or something more? ⭐⭐⭐⭐⭐

- **Ideal Answer:** Both — it provides mutual exclusion (only one thread in the critical section at a time) AND a happens-before guarantee: everything written before releasing the lock is visible to the next thread that acquires the same lock. Many candidates forget the visibility half.

- **Difficulty:** Advanced

Part 20 — Performance Optimization

Technique	Why It Helps
StringBuilder for repeated concatenation	Avoids creating a new String object per `+=` in loops
Pre-size collections (`new ArrayList<>(n)` , `new HashMap<>(n)`)	Avoids repeated resize/rehash operations
Avoid unnecessary object creation	Fewer allocations = less GC pressure and pause time
Primitive streams (`IntStream` , `LongStream`)	Avoids autoboxing overhead in numeric-heavy operations
Caching	Avoids redundant expensive computation/DB/network calls
Favor immutability	Immutable objects are inherently thread-safe and safely shareable/cacheable
Object pooling (brief)	Reuse expensive-to-create objects (e.g., DB connections) — but modern GC is fast enough that pooling plain objects is rarely worth the complexity anymore; mainly relevant for genuinely expensive resources
Stream vs loop	Plain loops often win for simple operations on small/medium data (less overhead); Streams shine for readability and parallelization on large datasets
Minimize boxing in hot paths	Autoboxing in tight loops creates throwaway wrapper objects
Lazy initialization	Defer expensive object creation until actually needed, avoiding wasted work on unused paths

```
// Lazy initialization example
class ExpensiveResource {
    private static volatile ExpensiveResource instance;
    static ExpensiveResource getInstance() {
        if (instance == null) {
            synchronized (ExpensiveResource.class) {
                if (instance == null) {                    // double-checked locking
                    instance = new ExpensiveResource();
                }
            }
        }
        return instance;
    }
}
```

Part 21 — Frequently Confused Concepts

Process vs Thread

Process	Thread
Own isolated memory	Shares memory with sibling threads
Expensive to create	Cheap to create

Runnable vs Callable

Runnable	Callable
No return value, no checked exceptions	Returns a value, can throw checked exceptions

synchronized vs ReentrantLock

synchronized	ReentrantLock
Implicit lock/unlock, auto-release	Explicit lock()/unlock(), manual (finally block)
No tryLock/fairness/interruptible wait	Supports all three

wait vs sleep

`wait()`	`sleep()`
Releases the object's lock	Does **not** release any lock
Must be called within `synchronized`	Can be called anywhere
Woken by `notify()` / `notifyAll()`	Wakes automatically after the timeout

notify vs notifyAll

`notify()`	`notifyAll()`
Wakes **one** arbitrary waiting thread	Wakes **all** waiting threads (they re-compete for the lock)
Risk: might wake the "wrong" thread for the condition	Safer default in most cases despite slightly more overhead

volatile vs AtomicInteger

`volatile`	`AtomicInteger`
Guarantees **visibility** only	Guarantees visibility **and** atomicity
`count++` is still NOT atomic even if volatile	`incrementAndGet()` is a single atomic CAS operation

Executor vs ExecutorService

`Executor`	`ExecutorService`
Just `execute(Runnable)` — bare minimum	Adds `submit()`, `Future`s, lifecycle management (`shutdown`), and more

submit vs execute

`execute()`	`submit()`
From `Executor`, takes `Runnable`, no result	From `ExecutorService`, takes `Runnable` / `Callable`, returns a `Future`

Future vs CompletableFuture

`Future`	`CompletableFuture`
Only blocking `.get()`	Non-blocking chaining (`thenApply`), combination (`thenCombine`), error handling

ArrayList vs CopyOnWriteArrayList

ArrayList	CopyOnWriteArrayList
Not thread-safe	Thread-safe, copies the array on every write
Fast writes	Slow writes (O(n) copy), fast/safe reads

HashMap vs ConcurrentHashMap

HashMap	ConcurrentHashMap
Not thread-safe	Thread-safe, fine-grained locking
Allows one null key	Disallows null entirely

Minor GC vs Major GC

Minor GC	Major GC
Collects Young Generation only	Collects Old Generation
Frequent, fast	Less frequent, slower

Heap vs Stack

Heap	Stack
Shared across threads	Per-thread
Stores objects	Stores local variables, references, call frames

Platform Thread vs Virtual Thread

Platform Thread	Virtual Thread
1:1 with OS thread, ~1MB+ stack	JVM-managed, M:N onto carriers, KB-scale
Thousands max practically	Millions possible

Parallel Stream vs CompletableFuture

Parallel Stream	CompletableFuture
Data-parallelism over a collection	General-purpose async task composition
Uses the shared common ForkJoinPool	Can use common pool or a custom `Executor`
Best for CPU-bound bulk data processing	Best for orchestrating independent async operations (I/O calls, API composition)

Part 22 — Real Project Usage

Use Case	Concepts Applied
Spring Boot / REST APIs	`@Async` methods backed by `ExecutorService` / `CompletableFuture`; thread pools sized per endpoint load
Payment Systems	`synchronized` / `ReentrantLock` around balance updates; idempotency via `ConcurrentHashMap`; careful deadlock-free lock ordering across accounts
Order Processing	`BlockingQueue`-based producer-consumer pipelines; `PriorityQueue` for order priority; retry logic with `CompletableFuture.exceptionally`
Notification Systems	`CompletableFuture.allOf` to fan out Email/SMS/Push notifications concurrently, then join
Batch Jobs	Fixed/scheduled thread pools; virtual threads for massively parallel I/O-bound batch tasks (e.g., calling thousands of external APIs)
Async APIs	`CompletableFuture` chains mapped directly to Spring WebFlux/reactive-style, or virtual-thread-per-request models in Java 21+
Caching	`ConcurrentHashMap` or dedicated caching libraries (Caffeine) for thread-safe in-memory caches; `volatile`/atomic flags for cache invalidation signals
Logging	Asynchronous loggers use background threads + queues so logging I/O doesn't block request-handling threads
High-Concurrency Systems	G1/ZGC tuning for pause-time SLAs; virtual threads for massive concurrent connection handling; careful JMM-aware design to avoid subtle visibility bugs

```
// Typical Spring Boot async pattern combining several Part topics
@Async
public CompletableFuture<OrderResult> processOrder(Order order) {
    return CompletableFuture.supplyAsync(() -> paymentService.charge(order))
        .thenApply(payment -> inventoryService.reserve(order))
        .exceptionally(ex -> OrderResult.failed(ex.getMessage()));
}
```

Part 23 — Placement Focus Summary ⭐⭐⭐⭐⭐

Topic	Importance	Why Companies Ask
HashMap concurrency issues (HashMap vs ConcurrentHashMap)	⭐⭐⭐⭐⭐	Combines Collections + Concurrency — a favorite "connect the dots" topic
`synchronized` & `volatile`	⭐⭐⭐⭐⭐	Foundation of Java concurrency — near-guaranteed in any backend interview
ExecutorService & thread pools	⭐⭐⭐⭐⭐	Directly relevant to real backend/Spring Boot work
Java Memory Model & happens-before	⭐⭐⭐⭐⭐	Separates candidates with real depth from surface-level API knowledge
Virtual Threads (Java 21)	⭐⭐⭐⭐⭐	Cutting-edge, high-signal topic — shows current Java awareness
G1 Garbage Collector	⭐⭐⭐⭐⭐	Default collector — expected baseline JVM knowledge
Deadlock prevention	⭐⭐⭐⭐⭐	Classic scenario-based question, tests practical reasoning
CompletableFuture	⭐⭐⭐⭐⭐	Modern async patterns used constantly in real Spring Boot codebases
Heap vs Stack, GC Roots	⭐⭐⭐⭐⭐	Foundational JVM memory model knowledge

Part 24 — Coding & Debugging Practice ⭐⭐⭐⭐⭐

(Problem prompts for self-practice.)

5 Synchronization Problems

1. Fix a race condition in a shared counter incremented by 10 threads using `synchronized`.
2. Convert a `synchronized` method to use `ReentrantLock` with proper `try/finally`.
3. Implement a thread-safe Singleton using double-checked locking with `volatile`.
4. Identify why `synchronized(this)` in two different methods of the same object can still deadlock with an external lock — fix it.

5. Implement a simple thread-safe bounded counter that blocks when incrementing past a max value (hint: `wait()` / `notifyAll()`).

5 Deadlock Problems

1. Reproduce a classic two-thread, two-lock deadlock, then fix it via lock ordering.
2. Given a 3-thread, 3-lock circular wait scenario, identify the cycle and redesign the locking order.
3. Fix a deadlock caused by acquiring a `ReentrantLock` and never releasing it in an exception path.
4. Detect a deadlock using a thread dump (describe what to look for).
5. Refactor a deadlock-prone design to use a single lock instead of nested locks.

5 Executor Problems

1. Submit 10 tasks to a fixed thread pool of size 3 and explain the execution order/timing.
2. Use `Future.get(timeout, unit)` to avoid blocking forever on a slow task.
3. Compare behavior of `newCachedThreadPool()` vs `newFixedThreadPool(5)` under a burst of 1000 short tasks.
4. Properly shut down an `ExecutorService` using `shutdown()` + `awaitTermination()`.
5. Build a scheduled task using `ScheduledExecutorService` that runs every 5 seconds.

5 CompletableFuture Problems

1. Chain three dependent async operations using `thenCompose`.
2. Combine two independent async calls using `thenCombine` and print the merged result.
3. Handle an exception in an async pipeline using `exceptionally`, providing a fallback value.
4. Use `allOf()` to wait for 5 parallel async calls to complete, then aggregate their results.
5. Use `anyOf()` to implement a "fastest response wins" pattern across 3 redundant service calls.

5 Output Prediction Questions

1. Predict the output of two threads incrementing a non-volatile, non-synchronized shared `int` 100,000 times each.
2. Predict whether a `volatile boolean` flag alone is sufficient to safely implement a counter increment across threads.
3. Predict the static initialization order for a class with interleaved static fields and static blocks.
4. Predict what happens when `wait()` is called outside a `synchronized` block.
5. Predict the output when `notify()` (not `notifyAll()`) is used with multiple waiting threads expecting different conditions.

5 Debugging Scenarios

1. An application intermittently hangs under load — thread dump shows two threads each `BLOCKED` waiting on a lock the other holds. Diagnose and fix.
2. A `HashMap` used across multiple threads occasionally produces incorrect results or infinite loops during resize — diagnose and fix.
3. A cached flag updated by a background thread is never seen by the main thread — diagnose (hint: missing `volatile`) and fix.
4. An `ExecutorService` is created per-request and never shut down, and the app eventually runs out of threads — diagnose and fix.
5. A virtual-thread-based service shows unexpectedly poor throughput — thread dump reveals many virtual threads pinned. Diagnose (hint: `synchronized` blocking calls) and fix.

Part 25 — Final Revision

Multithreading Cheat Sheet

- **Thread creation:** prefer `Runnable` / `Callable` + Executor over extending `Thread`.
- **Thread lifecycle:** NEW → RUNNABLE → (BLOCKED/WAITING/TIMED_WAITING) → TERMINATED.
- `synchronized`**:** implicit lock, auto-release, reentrant, provides mutual exclusion **and** happens-before visibility.
- `wait()` / `notify()` / `notifyAll()`**:** must be called inside `synchronized`; `wait()` releases the lock, `sleep()` does not.
- `volatile`**:** guarantees visibility and ordering, **not** atomicity (`count++` is still unsafe).
- **Atomic classes:** CAS-based, lock-free, atomic read-modify-write (`incrementAndGet()`).
- **Deadlock's 4 conditions:** mutual exclusion, hold-and-wait, no preemption, circular wait — break any one to prevent it.
- **ExecutorService:** `submit()` returns a `Future`; always `shutdown()` when done.
- **Virtual threads:** JVM-managed, cheap, great for I/O-bound concurrency; `synchronized` pins the carrier thread.

JVM Cheat Sheet

- **JVM = Class Loader + Runtime Data Areas + Execution Engine.**
- **Heap** (shared, objects) vs **Stack** (per-thread, locals/frames) vs **Metaspace** (class metadata, native memory).
- **Class loading:** Loading → Verification → Preparation → Resolution → Initialization.
- **Static fields** get default values in Preparation, real values in Initialization (source order).
- **JIT compiler:** compiles "hot" bytecode to native machine code at runtime.

Garbage Collection Cheat Sheet

- **Heap = Young Gen (Eden + 2 Survivor spaces) + Old Gen.**
- **Minor GC** = Young Gen only (frequent, fast). **Major/Full GC** = Old Gen / whole heap (rare, slow).
- **GC Root reachability** determines what's alive — not reference counting.
- **G1** = default since Java 9, region-based, balances throughput and pause time.
- **ZGC/Shenandoah** = ultra-low-pause collectors for large heaps/latency-critical apps.

Happens-Before Cheat Sheet

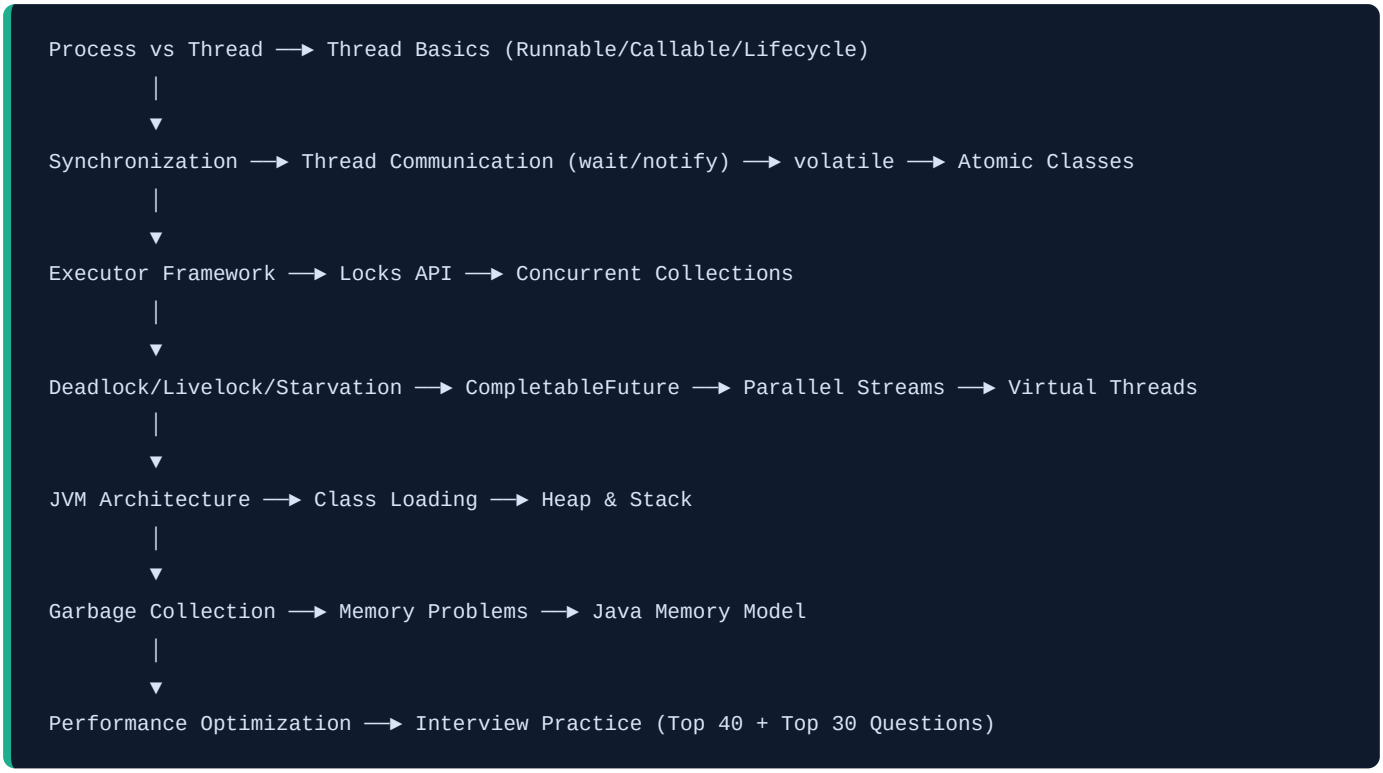
- Program order (within one thread) always happens-before the next statement.
- `unlock` happens-before the next `lock` on the **same** monitor.
- A `volatile` write happens-before every subsequent read of that field.
- `Thread.start()` happens-before anything in the new thread.
- Everything in a thread happens-before another thread's successful `join()` on it.

Memory Tricks

- **Heap vs Stack:** "Heap is shared housing, Stack is your personal locker."
- **Minor vs Major GC:** "Minor = tidy your desk often (Young Gen); Major = spring clean the whole house (Old Gen)."
- **volatile vs Atomic:** "volatile makes writes VISIBLE, Atomic makes read-modify-write SAFE."
- **Deadlock prevention:** "Same order, no deadlock" (consistent global lock ordering).

- **Virtual threads:** "Millions of virtual threads, few real (carrier) threads doing the actual work."

Topic Dependency Map



20-Minute Last-Minute Revision Checklist

- [ ] Can you explain the thread lifecycle diagram from memory?
- [ ] Can you explain what `synchronized` guarantees beyond mutual exclusion?
- [ ] Can you explain `wait()` vs `sleep()`?
- [ ] Can you explain what `volatile` guarantees and what it doesn't?
- [ ] Can you explain the four deadlock conditions and one prevention technique?
- [ ] Can you explain how ConcurrentHashMap achieves thread safety?
- [ ] Can you explain how virtual threads avoid wasting OS threads on blocking I/O?
- [ ] Can you draw the JVM's runtime data areas from memory?
- [ ] Can you explain Minor vs Major vs Full GC?
- [ ] Can you explain "happens-before" in one sentence?

Top 40 Concurrency Interview Questions

1. **Process vs Thread?** — Process has isolated memory; threads share memory within a process.
2. **Why prefer Runnable over extending Thread?** — Avoids using up single inheritance; decouples task from thread mechanism.
3. **Runnable vs Callable?** — Callable returns a value and can throw checked exceptions; Runnable can't.
4. **What does start() do that run() doesn't?** — start() creates a real new thread; run() alone just executes on the current thread.
5. **Explain the thread lifecycle states.** — NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
6. **What is a race condition?** — When multiple threads access/modify shared data concurrently without synchronization, causing unpredictable results.
7. **What does synchronized guarantee?** — Mutual exclusion AND happens-before visibility of prior writes.

8. **Object lock vs class lock?** — Instance-level synchronized methods lock on `this`; static synchronized methods lock on the Class object — independent locks.
9. **Is synchronized reentrant?** — Yes — a thread already holding a lock can re-acquire it without deadlocking itself.
10. **wait() vs sleep()?** — wait() releases the lock and must be in synchronized code; sleep() doesn't release any lock.
11. **notify() vs notifyAll()?** — notify() wakes one arbitrary waiting thread; notifyAll() wakes all of them.
12. **What is the Producer-Consumer problem?** — Coordinating producer and consumer threads sharing a bounded buffer, using wait/notify or BlockingQueue.
13. **What does volatile guarantee?** — Visibility and ordering of a single variable's reads/writes across threads.
14. **What does volatile NOT guarantee?** — Atomicity of compound operations like count++.
15. **What is CAS (Compare-And-Set)?** — An atomic hardware-supported operation: update a value only if it still equals an expected value, forming the basis of lock-free programming.
16. **Why use AtomicInteger over synchronized for a counter?** — Lower overhead, lock-free CAS-based atomic increments, no blocking.
17. **Why do thread pools exist?** — Avoid the overhead of creating/destroying threads per task; reuse a bounded set of worker threads.
18. **submit() vs execute()?** — submit() (ExecutorService) returns a Future and accepts Callable/Runnable; execute() (Executor) is fire-and-forget, Runnable only.
19. **shutdown() vs shutdownNow()?** — shutdown() lets running/queued tasks finish; shutdownNow() attempts immediate interruption and returns unstarted tasks.
20. **Fixed vs Cached thread pool?** — Fixed has a constant number of threads; Cached grows unboundedly and reuses idle threads, risky under sustained load.
21. **ReentrantLock vs synchronized?** — ReentrantLock offers tryLock, fairness, interruptible waits; synchronized is simpler and auto-releasing.
22. **What's the risk of ReentrantLock?** — Forgetting unlock() in a finally block causes a permanent lock leak.
23. **What does ReadWriteLock provide?** — Concurrent multiple readers, exclusive single writer — better read-heavy throughput.
24. **How does ConcurrentHashMap avoid locking the whole map?** — Fine-grained per-bucket locking plus CAS operations.
25. **When would CopyOnWriteArrayList hurt performance?** — Write-heavy workloads — every write copies the entire array.
26. **What is BlockingQueue used for?** — Implementing Producer-Consumer patterns with automatic blocking on full/empty.
27. **What are the four conditions for deadlock?** — Mutual exclusion, hold-and-wait, no preemption, circular wait.
28. **How do you prevent deadlock?** — Consistent global lock ordering across all threads.
29. **Deadlock vs livelock vs starvation?** — Deadlock = stuck waiting; livelock = actively responding but no progress; starvation = never getting CPU/lock access.
30. **Why was CompletableFuture introduced?** — To support composable, non-blocking asynchronous pipelines, unlike plain Future's blocking-only get().
31. **thenApply vs thenCompose?** — thenApply transforms a result; thenCompose chains another async operation, avoiding nested futures.
32. **allOf() vs anyOf()?** — allOf completes when all given futures complete; anyOf completes when the first one does.
33. **When should you use parallel streams?** — Large datasets, CPU-heavy, independent operations, order doesn't matter.
34. **Why are parallel streams risky for I/O-bound work?** — They share the common ForkJoinPool; blocking tasks can starve unrelated parallel work app-wide.
35. **What problem do virtual threads solve?** — Enabling millions of lightweight, JVM-managed concurrent tasks without the memory/scheduling cost of that many OS threads.
36. **Why does blocking I/O not waste resources with virtual threads?** — The JVM unmounts the blocked virtual thread from its carrier, freeing the carrier for other work.
37. **What is the "pinning" problem with virtual threads?** — A virtual thread blocking inside a synchronized block can't unmount, blocking its carrier thread too.
38. **Are virtual threads good for CPU-bound work?** — No — they help with I/O-bound concurrency, not raw computation speed.

39. **What's the difference between a platform thread and a virtual thread in terms of scale?** — Platform threads: thousands max practically; virtual threads: millions possible.
40. **How would you diagnose a deadlock in production?** — Take a thread dump and look for threads in BLOCKED state each waiting on a lock held by another thread in the cycle.

Top 30 JVM Interview Questions

1. **What are the main components of the JVM?** — Class Loader Subsystem, Runtime Data Areas, Execution Engine.
2. **What replaced PermGen and why?** — Metaspace (Java 8+) — grows automatically in native memory, avoiding PermGen's fixed-size OOM issue.
3. **Heap vs Stack?** — Heap is shared, stores objects; Stack is per-thread, stores locals/frames.
4. **What is the PC Register?** — Per-thread pointer to the currently executing JVM instruction.
5. **What does the JIT compiler do?** — Compiles frequently executed bytecode into native machine code at runtime for speed.
6. **What are the five phases of class loading?** — Loading, Verification, Preparation, Resolution, Initialization.
7. **What happens during Preparation?** — Static fields get default values (0/null/false), not their real initializers yet.
8. **What order do static blocks/fields run in?** — Top-to-bottom, exactly as they appear in source code.
9. **What is a GC Root?** — A starting reference point (stack locals, static fields, active threads, JNI refs) used to determine object reachability.
10. **Why does the heap have generations?** — Most objects die young — frequent cheap collection of a small Young Gen is more efficient than scanning the whole heap.
11. **Minor GC vs Major GC vs Full GC?** — Minor = Young Gen; Major = Old Gen; Full = entire heap (and often Metaspace) — slowest.
12. **What is the default garbage collector since Java 9?** — G1 (Garbage First).
13. **How does G1 differ from older collectors?** — Divides heap into many regions, prioritizes collecting the most garbage-heavy regions first, offering predictable pause times.
14. **When would you choose ZGC?** — Ultra-low-latency requirements with very large heaps.
15. **What causes OutOfMemoryError: Java heap space?** — The heap is full and GC can't free enough memory to satisfy an allocation.
16. **What causes StackOverflowError?** — Excessive/unterminated recursion exhausting a thread's stack.
17. **How can a memory leak happen with automatic GC?** — An object stays reachable (e.g., in a growing static collection) even though it's logically no longer needed.
18. **What is a ClassLoader leak?** — A ClassLoader can't be collected because something still references one of its loaded classes, common in app-server hot redeploys.
19. **What is the Java Memory Model (JMM)?** — The specification defining visibility, ordering, and atomicity guarantees for shared memory across threads.
20. **What is "happens-before"?** — A guarantee that if A happens-before B, A's memory effects are visible to B.
21. **Does synchronized only provide mutual exclusion?** — No — also a happens-before/visibility guarantee across lock release/acquire.
22. **What guarantees does volatile provide?** — Visibility and ordering for that variable, not atomicity of compound operations.
23. **Why might two threads see different values of a non-volatile shared variable?** — CPU-local caching means writes aren't guaranteed to be immediately visible without a happens-before edge.
24. **What is escape analysis?** — A JIT optimization that can allocate certain objects on the stack instead of the heap if they're proven not to escape a method, reducing GC pressure.
25. **What does -Xmx configure?** — The maximum heap size for the JVM.
26. **What's the difference between the heap and Metaspace?** — Heap stores objects (GC-managed); Metaspace stores class metadata (grows in native memory).
27. **Can an object be garbage collected while a reference to it still exists on some thread's stack?** — No — any reachable reference (including from an active stack frame, a GC Root) keeps it alive.

28. **Why is object allocation usually fast in Java despite GC overhead?** — Eden space allocation is typically just a pointer bump; the real cost is paid later during collection, not at allocation time.
29. **What's the practical impact of a Full GC pause on a production service?** — Application threads stop entirely during the pause ("stop-the-world"), potentially causing request timeouts — a key reason for choosing low-pause collectors like G1/ZGC.
30. **How would you start investigating a suspected memory leak in production?** — Take a heap dump, analyze it (e.g., with Eclipse MAT) for unexpectedly large retained object graphs, and trace their GC Root reachability path.

Key Takeaways

- **Concurrency bugs are about shared mutable state** — every tool in this handbook (synchronized, volatile, locks, atomics, concurrent collections) exists to manage that safely.
- **`volatile` ≠ thread-safe for compound operations** — it guarantees visibility, not atomicity; know the difference cold.
- **Virtual threads are the biggest recent shift in Java concurrency** — understand the carrier-thread model and the `synchronized` pinning gotcha.
- **HashMap internals + concurrency is the highest-leverage combined topic** — expect interviewers to connect Handbook 3's HashMap knowledge with this handbook's concurrency knowledge.
- **G1 is the default GC for a reason** — know the generational heap model and why region-based collection improves on older stop-the-world designs.

End of Handbook 4. This completes the "Java Placement Master Series" — Handbooks 1–4 together cover Java Foundations, OOP, Core Java, Collections/Modern Java, and Concurrency/JVM/Performance for full-spectrum placement readiness.